

Slack Integration Guide – PinionAI Agent Runner

This guide covers Slack app setup, local development, and production deployment of the Slack bot implemented in `chat_slack.py`.

Table of Contents

1. [Slack App Configuration](#)
 2. [Environment Setup](#)
 3. [Local Deployment](#)
 4. [Production Deployment on a VM](#)
 5. [Usage Guide](#)
 6. [Troubleshooting](#)
 7. [Full GCP VM Deployment Example](#)
-

1. Slack App Configuration

To use the Slack integration, create and configure a Slack App in the [Slack API Dashboard](#).

A. Create the App

1. Click **Create New App > From scratch**.
2. Name your app (for example, "PinionAI Bot") and select your workspace.

B. Enable Socket Mode

1. In the left sidebar, go to **Settings > Socket Mode**.
2. Toggle **Enable Socket Mode** to **On**.
3. Generate an **App-level token** with the `connections:write` scope.
4. Copy this token as `SLACK_APP_TOKEN`.
5. Enable **Interactivity & Shortcuts**, **Slash Commands**, and **Events**.

App Hosting Note: When the bot runs in Slack Socket Mode, no public inbound Slack webhook endpoint is needed. A hosting VM only needs outbound HTTPS traffic to Slack.

C. Configure Bot Scopes

1. Go to **Features > OAuth & Permissions**.
2. Add these bot scopes:
 - `chat:write`
 - `files:read`
 - `im:history`
 - `channels:history`
 - `groups:history`
3. Install the app to your workspace.
4. Copy the **Bot User OAuth Token** as `SLACK_BOT_TOKEN`.

D. Enable Events

1. Go to **Features > Event Subscriptions**.
2. Enable events.
3. Subscribe to:
 - `message.channels`
 - `message.groups`
 - `message.im`

E. Enable the Messages Tab

1. Go to **Features > App Home**.
2. Ensure **Messages Tab** is enabled.
3. Allow users to send slash commands and messages from the Messages tab.

F. Add Slash Commands

1. Go to **Features > Slash Commands**.
2. Create `/end` with a short description such as "Ends the PinionAI chat session".

2. Environment Setup

Use a local `.env` file for development and a server-side env file or Docker env file for production.

Example local environment:

```
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
agent_id=<YOUR_AGENT_ID>
host_url=https://api.pinionai.com

SLACK_BOT_TOKEN=xoxb-...
SLACK_APP_TOKEN=xapp-...
```

- For production, the same values can be placed in `deploy/prod-slack/env.yaml` or converted into a `.env` file for a VM deployment.

What each variable means

- `client_id` / `client_secret`: Your PinionAI Studio API credentials.
- `agent_id`: The ID of the agent you want to serve.
- `host_url`: The PinionAI API host.
- `SLACK_BOT_TOKEN`: Slack bot token used for posting messages.
- `SLACK_APP_TOKEN`: Slack app-level token used for Socket Mode.

Important Note: When hosting a Slack Agent you can serve a specific agent configuration, and also drop AIA (AI Agent) files in the conversation to add agent specific functionality. IF you do not want to start with

a specific agent, but only desire to allow agent capabilities to be merely added by files, `client_id`, `client_secret` and `agent_id` are **NOT Required** variables.

3. Local Deployment

1. Activate your environment:

```
source pinionai-streamlit/bin/activate
```

2. Install dependencies:

```
uv pip compile requirements.in -o requirements.txt
uv pip sync requirements.txt
```

3. Run the bot:

```
python chat_slack.py
```

4. Production Deployment on a VM

A VM is the recommended production hosting option for this Slack bot because it keeps a long-running Socket Mode connection alive.

4.1 Build the Slack-specific container image

Use the Slack-specific Dockerfile:

```
docker build -f Dockerfile.slack -t pinionai-slack:latest .
```

4.2 Prepare the environment file

On the VM, create a `.env` file with the same values you would normally put in `deploy/prod-slack/env.yaml`.

Example:

```
host_url=https://api.pinionai.com
client_id=<YOUR_CLIENT_ID>
client_secret=<YOUR_CLIENT_SECRET>
agent_id=<YOUR_AGENT_ID>
```

```
SLACK_BOT_TOKEN=xoxb-...  
SLACK_APP_TOKEN=xapp-...
```

What each variable means

- `client_id` / `client_secret`: Your PinionAI Studio API credentials.
- `agent_id`: The ID of the agent you want to serve.
- `host_url`: The PinionAI API host.
- `SLACK_BOT_TOKEN`: Slack bot token used for posting messages.
- `SLACK_APP_TOKEN`: Slack app-level token used for Socket Mode.

Important Note: When hosting a Slack Agent you can serve a specific agent configuration, and also drop AIA (AI Agent) files in the conversation to add agent specific functionality. IF you do not want to start with a specific agent, but only desire to allow agent capabilities to be merely added by files, `client_id`, `client_secret` and `agent_id` are **NOT Required** variables.

4.3 Run the container on the VM

```
docker run -d \  
  --name pinionai-slack \  
  --restart unless-stopped \  
  --env-file /path/to/.env \  
  pinionai-slack:latest
```

Check logs with:

```
docker logs -f pinionai-slack
```

4.4 Google Compute Engine VM

1. Create a Debian 12 or Ubuntu 22.04 VM in GCP.
2. Allow SSH and optionally HTTP/HTTPS traffic.
3. Install Docker on the VM.
4. Copy the repository to the server.
5. Build the image using `Dockerfile.slack`.
6. Start the container as shown above.

Example install steps on Debian/Ubuntu:

```
sudo apt update  
sudo apt install -y docker.io  
sudo systemctl enable docker  
sudo systemctl start docker  
sudo usermod -aG docker "$USER"
```

4.5 DigitalOcean Droplet

1. Create a Debian 12 or Ubuntu 22.04 droplet.
2. SSH into the droplet.
3. Install Docker.
4. Clone the repository and build the Slack image.
5. Run the container with your `.env` file.

Example:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker
sudo systemctl start docker
```

4.6 Optional systemd service

If you prefer process management over Docker, run the bot directly from a Python virtual environment with a systemd unit. This is a good fit for long-running production use.

5. Usage Guide

Chatting

- **Direct Message:** Send a message to the bot in its Messages tab.
- **Channels:** Invite the bot to a channel and type a message.

Dynamic Agent Loading

You can change the active agent for a channel by uploading a `.aia` file.

1. Upload the `.aia` file to the chat.
2. The bot will switch to that agent for subsequent messages in that channel.
3. If the file is private, the bot will ask for the `key_secret`.

Cleaning Slack Formatting

The bot removes Slack formatting wrappers such as `<mailto:...>` before sending text to the AI.

Ending a Session

To clear the active session:

- Use `/end`, or
- Send `!end` as a message.

6. Troubleshooting

- **"Sending messages to this app has been turned off"**: Enable the Messages tab and user messaging in App Home.
 - **Bot does not reply in channels**: Ensure the bot is invited to the channel and that `message.channels` is subscribed.
 - **Bot does not see messages**: Check event subscriptions and app scopes.
 - **Module import errors**: Make sure the virtual environment is activated and that dependencies are installed.
-

7. Full GCP VM Deployment Example

FULL INSTRUCTIONS: Deploy `chat_slack.py` on Free GCP `e2-micro` VM tier in `us-central1`

This guide explains how to deploy the PinionAI Slack bot implementation in `chat_slack.py` on a Google Cloud Platform `e2-micro` VM instance in `us-central1`.

It assumes the bot runs in Slack Socket Mode, so no public inbound Slack webhook endpoint is needed. The VM only needs outbound HTTPS traffic to Slack.

1. Create the VM instance

1. Open Google Cloud Console.
2. Navigate to `Compute Engine` → `VM instances` → `Create instance`.
3. Set the following values:
 - Name: `pinionai-slack-bot`
 - Region: `us-central1`
 - Zone: choose any `us-central1-*` zone (for example `us-central1-a`).
 - Machine type: `e2-micro` (free-tier eligible).
 - Boot disk: use `Debian 12` (Bookworm) or the latest stable Debian image.
4. Under "Identity and API access", leave default scopes or use the minimum required.
5. Under "Firewall", you do not need to enable HTTP/HTTPS for the Slack bot itself because it uses Socket Mode.

GCP FREE TIER

To stay within the free tier for your Google Cloud `e2-micro` instance, you must carefully configure your storage settings to avoid unexpected charges:

- Boot Disk Type: Standard Persistent Disk
- Storage Size: Maximum of 30 GB
- Operating System: Debian only. Choose Debian GNU/Linux 12 (Bookworm) or newer for best compatibility.

To ensure your setup remains completely free, follow these recommended settings: [3]

- Region: Select `us-central1`, `us-west1`, or `us-east1`.
- Machine Type: Series E2, Machine type `e2-micro`.
- Data Protection: Uncheck or select No backups (snapshots cost extra).

- Observability: Uncheck Install Ops Agent (this consumes your limited memory and can incur costs).

You can manage and create your instance using the [Google Cloud Console](#). Review the [Google Cloud Free Tier](#) for full terms and regional limits.

This guide is written for Debian, so proceed with Debian-specific steps and commands.

Use these exact settings during creation to ensure you stay within the Google Cloud Free Tier:

Compute Engine Settings

- Region: Choose us-central1 (Iowa), us-west1 (Oregon), or us-east1 (S. Carolina).
- Series: E2
- Machine Type: e2-micro (2 vCPUs, 1 GB memory)

Boot Disk Settings

Click Change under the "Boot disk" section and configure:

- Operating System: Debian
- Version: Debian GNU/Linux 12 (Bookworm) or latest stable Debian release
- Boot Disk Type: Standard persistent disk (Do NOT choose Balanced or SSD)
- Size: 30 GB (Maximum free tier limit)

Why **e2-micro** is acceptable

- **e2-micro** provides 1 vCPU and 1 GB RAM.
- The Slack bot is a lightweight long-running Python process.
- The bot does not require a large server unless your workload is heavy or you also run other services.

Note: If you plan to host additional services on the same VM, choose a larger machine type.

2. Access the VM via SSH

You can connect to the VM using either method:

- Add your SSH public key to the VM when creating it, then connect with your local SSH client.
- Use the Cloud Console browser SSH button if you prefer not to manage SSH keys locally.

Option A: Use an SSH key

1. In the VM creation page, open the "Security" or "SSH Keys" section.
2. Paste your public SSH key from `~/.ssh/id_rsa.pub` or another key.
3. Save and create the VM.
4. Connect from your local terminal:

```
gcloud compute ssh pinionai-slack-bot --zone=us-central1-a
```

Option B: Use browser SSH

1. After the VM is created, click the **SSH** button in the Google Cloud Console.
2. The browser will open a shell session automatically.
3. This is useful if you do not want to set up SSH keys locally.

3. FREE TIER OPTIMIZATIONS - Post-Setup Steps

Because the e2-micro only has 1 GB of RAM, Debian can run out of memory quickly. Run these commands right after connecting via SSH to optimize your system:

1. Update your system:

```
sudo apt update && sudo apt upgrade -y
```

2. Create a Swap File (Crucial for 1GB RAM): This uses 2 GB of your 30 GB storage as virtual memory to prevent crashes.

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

3. Install required packages

Once connected to the VM, update packages and install Python, Git, and build tools.

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip git curl build-essential
```

Why these packages are needed

- **python3**, **python3-venv**, and **python3-pip** are required to create a Python virtual environment and install dependencies.
- **git** is required to clone the repository.
- **curl** is useful for troubleshooting and downloading files.
- **build-essential** ensures any Python dependency with native extensions can compile.

4. Clone the repository

Clone the PinionAI Slack bot repo into your home directory.

```
cd /home/$USER
git clone https://github.com/pinionai/pinionai-streamlit-agent.git
cd pinionai-streamlit-agent
```

If you plan to keep the repo updated, you can later run `git pull` in this directory.

5. Create and activate a Python virtual environment

Create an isolated Python environment so dependencies do not interfere with the system Python.

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Why use a virtual environment

- Keeps project dependencies isolated.
 - Makes it easier to manage upgrades.
 - Prevents conflicts with system packages.
-

6. Create the `.env` configuration file

Copy the provided example file and set your Slack and PinionAI credentials.

```
cp .env.example .env
chmod 600 .env
nano .env
```

Fill in values like:

```
client_id=<YOUR_CLIENT_ID_HERE>
client_secret=<YOUR_CLIENT_SECRET_HERE>
agent_id=<YOUR_AGENT_ID_HERE>
host_url=https://api.pinionai.com

SLACK_BOT_TOKEN=xoxb-...
SLACK_APP_TOKEN=xapp-...
```

What each variable means

- `client_id` / `client_secret`: Your PinionAI Studio API credentials.
- `agent_id`: The ID of the agent you want to serve.
- `host_url`: The PinionAI API host.
- `SLACK_BOT_TOKEN`: Slack bot token used for posting messages.
- `SLACK_APP_TOKEN`: Slack app-level token used for Socket Mode.

Keep `.env` private and do not commit it to Git.

Important Note: When hosting a Slack Agent you can serve a specific agent configuration, and also drop AIA (AI Agent) files in the conversation to add agent specific functionality. IF you do not want to start with a specific agent, but only desire to allow agent capabilities to be merely added by files, `client_id`, `client_secret` and `agent_id` are **NOT Required** variables.

7. Configure your Slack app correctly

Your Slack app must be set up for Socket Mode.

Required Slack app settings

- Enable Socket Mode in "Socket Mode" settings.
- Create an App-Level Token with the `connections:write` scope.
- Install the app into your workspace.
- In "OAuth & Permissions", ensure the bot has `chat:write` and `files:read` permissions.

Recommended Slack permissions

- `chat:write` – allows the bot to send messages.
- `files:read` – allows the bot to access uploaded `.aia` files.
- `connections:write` – required for Socket Mode.

8. Test the Slack bot manually

Run the bot directly from the repository root.

```
source .venv/bin/activate
python chat_slack.py
```

If the bot starts successfully, it should connect to Slack and remain running in the terminal. If there is a problem, check the error output and confirm that your `.env` file values are correct.

What to expect

- The bot will connect to Slack using Socket Mode.
- It will listen for messages and file uploads in the workspace.
- If no agent is loaded, it may ask you to upload an `.aia` file or configure `agent_id`.

9. Install the bot as a `systemd` service

Create a service so the bot starts automatically and restarts if it fails.

echo \$USER

replace the output from the echo where \$USER is below for file content.

```
sudo nano /etc/systemd/system/pinionai-slack.service
```

Paste this file content:

```
[Unit]
Description=PinionAI Slack Bot
After=network.target

[Service]
WorkingDirectory=/home/$USER/pinionai-streamlit-agent
EnvironmentFile=/home/$USER/pinionai-streamlit-agent/.env
ExecStart=/home/$USER/pinionai-streamlit-agent/.venv/bin/python
/home/$USER/pinionai-streamlit-agent/chat_slack.py
Restart=always
RestartSec=5
User=$USER
Group=$USER
StandardOutput=append:/var/log/pinionai-slack.log
StandardError=append:/var/log/pinionai-slack.err.log

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl daemon-reload
sudo systemctl enable pinionai-slack
sudo systemctl start pinionai-slack
sudo systemctl status pinionai-slack
```

Why use `systemd`

- Automatically restarts the bot if it crashes.
- Starts the bot when the VM boots.
- Makes management easier with `systemctl`.

10. Monitor the bot and logs

Use `journalctl` to watch the service logs in real time.

```
sudo journalctl -u pinionai-slack -f
```

If you need to restart after changing `.env`, run:

```
sudo systemctl restart pinionai-slack
```

If the service fails to start, inspect the logs for configuration or dependency errors.

11. Network considerations

- Slack Socket Mode only requires outbound HTTPS traffic.
 - You do not need to open inbound ports for the Slack connection itself.
 - If you add a separate web service later, open ports `80` and `443` only as needed.
-

12. Commands summary

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip git curl build-essential
cd /home/$USER
git clone https://github.com/pinionai/pinionai-streamlit-agent.git
cd pinionai-streamlit-agent
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
cp .env.example .env
nano .env
python chat_slack.py
```

Then create the `systemd` service and enable it.

13. Optional: using browser SSH

If you prefer not to use local SSH keys, the Google Cloud Console browser SSH option is a good alternative.

- Click `SSH` next to your VM in the console.
 - A browser window opens a shell directly in the VM.
 - This is useful for one-off setup or when you do not want to manage `~/.ssh` keys locally.
-

14. Important notes for e2-micro

- Keep the VM dedicated to the Slack bot for best stability.
- Do not run resource-heavy processes at the same time.
- If the VM becomes too slow, consider upgrading to e2-small or larger.
- Always keep your tokens and keys secure.